AR 環境における 3D モデルの操作・インタラクション

Manipulation and Interaction of 3D Models in an AR Environment

5420　チェット（指導教員　秋本 高明）

Keywords: ARToolKit, 3DCG, Programming Techniques

Abstract

The use of augmented reality (AR) technologies has steadily been growing in various fields, including design, education, gaming, and beyond. AR offers unique possibilities in creating and manipulating 3D virtual environments overlayed on the physical world. This research seeks to explore these potentials through the development of a program using ARToolkit and other software that allow for exploration of techniques for manipulate and interaction of 3D models in real time. The goal is to develop a system that enables real-time advanced and smooth manipulation and interaction with 3D CG (Computer Graphic) in augmented reality. Furthermore, as a practical application of this system, we aim to create a basic augmented reality game.

1. Introduciton

Augmented Reality (AR) is a technology that's changing how we see the world by adding digital elements to our real surroundings. Think of AR glasses like Google's, which can translate languages in real-time, making it easier to talk to people who speak different languages. AR lets you see and interact with things like 3D models and information right in front of you, as if they were there.

People are getting more and more interested in AR because of its potential to revolutionize various sectors from education to design to gaming. It mixes virtual things into our real world, making our daily interactions with technology more interactive and fun. As AR technology gets better, it's opening new ways for us to use digital content in our everyday lives.

However, current AR technology faces several challenges. Notably, the inability to physically touch and feel digital overlays, limitations in accurately of spatial awareness, and the imperfect interaction between virtual and real-world objects. This highlights the need for further research and development in AR technology.

Our research is focused on leveraging AR to develop a system for advanced and seamless interaction with 3D computer graphics in real-time environments. We will explore the capabilities of AR technology, with the goal of enhancing AR applications, making them more responsive and user-friendly. Additionally, we intend to apply this knowledge to create a straightforward AR game, demonstrating the practical utility and versatility of our approach. Through this research, we aim to contribute to the evolving field of AR, enhancing the quality and effectiveness of augmented experiences.

2. Theory

2.1 ARToolKit

ARToolKit is a set of C/C++ open-source libraries, which has widely been used for coding programs for AR applications. The main functions provided by ARToolKit are acquisition of images from a camera, detection of markers (example, “hiro.patt” as shown in fig 1.) and recognition of patterns, measurement of 3D positions and orientations of markers, and display of 3D CG composites on the actual images. GLUT (OpenGL Utility Toolkit) is used for drawing camera images and setting viewpoints in virtual space. To program AR applications, we’re using Visual Studio 2022 as a C++ language program development environment.

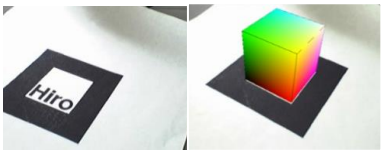


Fig 1. Example of ARToolKit program

2.2 Overview of program by ARToolKit

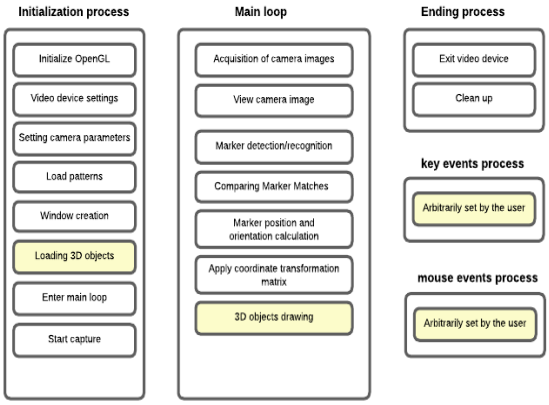


Fig 2：Overview of program by ARToolKit

The initial function performs initialization processing and starts the main loop function. After that, the main loop repeats endlessly. In the main loop, a marker is detected from the acquired image, the 3D position and orientation of the marker is calculated, the marker is converted to a coordinate system with the marker center as the origin, and a 3D object is drawn. While the main loop is running, keyboard and mouse input can be accepted. It is common to end the process by key input.

Most of the processes shown in white blocks are standard processes, so they can be used directly from the sample code provided by ARToolKit. The main part that the programmer can create is ``How and when models should be displayed'. This allows us to build 3D models manipulation and interaction. ARToolKit uses "OpenGL" for 3D graphics processing. In other words, knowledge of OpenGL is required to draw and manipulate 3D objects. This opens a realm of possibilities for imaginative and interactive programming with 3D objects in augmented reality.

3. Programming 3D models manipulation

3.1 Create and display a 3D model

In this research project, we used Metasequoia 4 for the 3D model's design. The initial stage of the research project involved programming marker detection, where ARToolKit recognizes a specific physical marker and overlays 3D models created in Metasequoia 4 onto the marker.

GLMetaseq library is an open-source toolkit specifically designed to render Metasequoia (.mqo) files. Therefore, we use GLMetaseq library to handle and display 3D models within our augmented reality environment. To prepare for the rendering of Metasequoia models, we first initialized the GLMetaseq toolkit by calling the function mqoInit(). This function ensures all the necessary procedures are undertaken to make GLMetaseq operational. The main task here is to load a Metasequoia 3D model into our program. This was accomplished using the function mqoCreateModel(), which reads the Metasequoia file, parses the contained 3D data, and stores it as a model object that can be displayed and manipulated within our application.

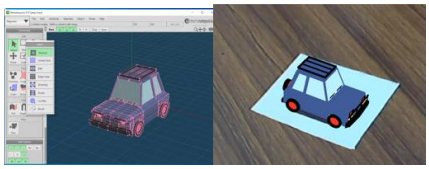


Fig 3. Create and display a 3D model

3.2 Manipulate a 3D model

The primary functionalities of the developed program encompass real-time control of these 3D models, including movement in various directions (forward, backward, left, right, up, and down), and the ability to resize and rotate the model.

In our program, we use a key event function which is a callback that captures keyboard input. It's used to set a controller state (for example, ‘W’ is pressed) that's used later in the calculating 3D model state function. The calculating 3D model state function uses the controller state to adjust the movement, rotation, and size of the 3D models. In this way, the program provides an interactive control mechanism where, for instance, the pressing of 'W' initiates forward movement, while the pressing of 'S' initiates backward movement of the 3D model.

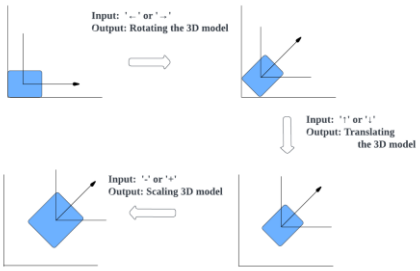


Fig 4. Example of adjusting 3D models' state

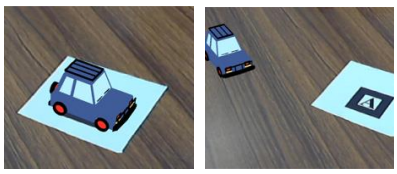


Fig 5. Manipulating 3D model position, rotation, and size

3.3 Generate more 3D models

The objective here is to generate specific 3D models when we need to. A common way to generate 3D models is text-command model generation. When a user inputs a specific word, such as "car", “bicycle”, and the corresponding 3D model will be called and generated, adding an interactive dimension to the AR environment. Model generation is handled within the created key event function. For example, when the 'i' key is pressed, the user is asked to enter a word (in this context, a model name). This input is compared to the existing model names stored in the model array. When a match is found, a new 3D model is created based on the corresponding model from the model array and added to the scene.

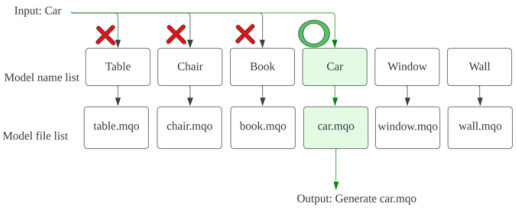


Fig 6. Process of finding 3D model by text input

Fig 7. Generating multiple 3D models

3.4 Selection and deletion

Subsequently, a selection function was implemented, enabling the ability to choose and manipulate a specific model among multiple 3D models within the AR environment. In this program, we use '>' and '<' keys for selection. Pressing '>' increments the current model index, cycling forward through the list of 3D models generated within the AR environment. If the end of the list is reached, it loops back to the first model. Conversely, pressing '<' decrements the model index, moving backwards. If the user tries to move back past the first model, the selection wraps around to the last model in the list. This functionality provides a simple way for users to select and focus on different 3D models within the AR environment. The selected 3D model, which the user is currently manipulating, can be programmed to look different from others, for example, illuminate brightly, draw a red circle at the bottom etc.

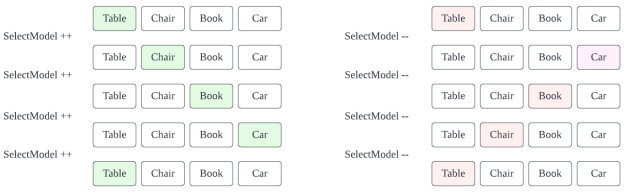


Fig 8. Example of selection process

The deletion mechanism in the program is designed to manage the array of 3D models effectively. When a user decides to delete a selected model, the program first removes the data at the selected index. Following this, it shifts every subsequent model up by one index to fill the gap. This means that the model previously at index 'n+1' moves to index 'n', the model at 'n+2' moves to 'n+1', and so

on. After all other models have been shifted, the data in the last index is cleared. This ensures that there is no duplicate or residual data left at the end of the array after the deletion process. This deletion mechanism is key for effectively managing the collection of models in the AR environment, allowing for the removal of models that are no longer needed, while keeping the remaining models properly sequenced.

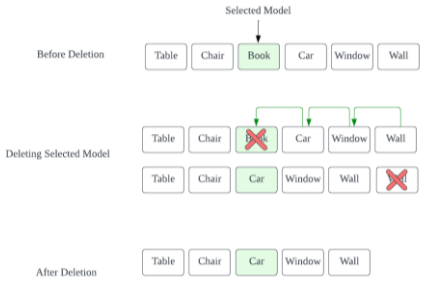


Fig 9. Example of deletion process

3.5 Design and reuse the models

We want to demonstrate how we can adjust the 3D models using the functionalities we programmed so far. This involved the creation of a detailed virtual room. Various 3D models were generated, and using the control functions of the program, these models were rearranged and positioned to resemble a well-organized room as shown in Fig 10. After designing the models, we could also store all the positions information of these 3D models that combine into this well-organized room into a file for later use. This opens a door for developing 3D models software design in real time in AR environments.

Fig 10. designing 3D models

3.6 Manipulate multiple models as singular

Fig 11 and 12 shows how we can reuse the pre-design models as a singular model. By calculating the geometric center of all the models, this point becomes a pivot or reference for the group. The program then determines the relative offset for each model from this central point. Consequently, any transformation applied to the center – be it moving, rotating, or scaling – is mirrored across all models, maintaining their respective positions and orientations. When the group is moved, the reference point (the center) is relocated to a new position. Each model then follows, maintaining its relative offset from the center. During rotation, the reference point acts as the axis of rotation. Each model rotates around this point, keeping its relative distance constant but changing its direction in relation to the center. Scaling involves increasing or decreasing the size of the models relative to the center point. When the group of models is scaled up or down, each model changes size at the same rate. To maintain their positions relative to the group's center, the offsets of each model from this central point are also scaled at the same rate. This ensures that, as the models grow or shrink, they retain their relative arrangement, preserving the group's overall structure. This method transforms the way models are managed in the AR space, shifting from individual adjustments to a more holistic, efficient control of the entire group.

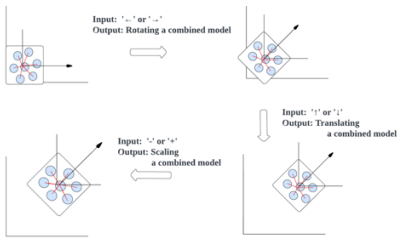


Fig 11. Manipulate multiple models as singular

Fig 12. Manipulate multiple models as singular

4. Programming 3D models interaction

4.1 3D models animations

The first thing we can do for 3D models' interactions is to create and display model animations. A way to do that is setting bones for models on Metasequoia 4. In that way, we can easily create sequences of simple poses (or frames), therefore able to create animation which is essentially a sequence of poses. To display this animation in a program, we use a function mqoCallSequence provided by GLMetaseq library. This function is designed to take the sequence of poses we've created and display them in order, making the model appear to move.

Fig 13. Creating poses and animations

4.2 Collision detection

For simplification, we consider 3D models as cylinders with their radius and height. We start by setting up appropriate radius and height for each 3D model according to its size. When do models contact each other? We can determine that “If the distance from the main model to another model is shorter than the total of both model's radius, it means that the models have made contact with each other.”. In fig 14, we show a simple illustration of checking model collision. In the program, we calculate the distance between two models, and both model's radius. Then, we check collision by comparing the distance and the radius. In this way, we enable model and model interaction with the simple concept of “when models are in contact, initiate some actions”.

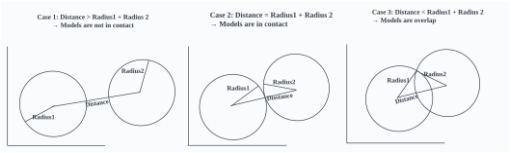


Fig 14. Illustration of model collision concept

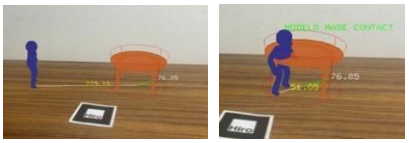


Fig 15. Models' collision detection program

4.3 Domain control

A simple interaction when model in contact is to not allow the primary model to run into other models' area. This can be achieved by implementing a simple rule, that is, “If the models made contact with each other, move one model back by the amount of overlap.”.

Implementing domain control, where a model is restricted from entering another model's space, is essential for defining boundaries and creating structured interactions in a virtual space. This control mechanism can be used to create obstacles, define territories, or establish rules within a game or simulation.

4.4 Pick and drop

Another simple interaction but quite important is pick and drop. When the primary model is in contact with a secondary model, we can initiate a pickup by right click on the mouse, allowing primary model to move around while holding a secondary model. While the primary model is holding the secondary model we can release or drop the secondary model with a left click on the mouse. Pick and drop are crucial for tasks that involve object manipulation, such as collecting, organizing, building, or sorting activities.

4.5 Jumping on or stacking

Another interaction is allowing models to jump on each other or stack. First, we can simulate a jump by setting up gravity. In the program loop, we constantly decrease a model’s Z coordination value until it reaches the minimum value (0 for ground level). When the spacebar is pressed, the selected model increases its Z coordinate with jumping velocity while also gradually decreasing by the set-up gravity. If the selected model touches a model and the selected model's Z coordinate is less than the height of that model, prevent the selected model from passing through that model (domain control at ground level). However, if the selected model is in the air and its Z value is higher than a model it touches, then gravity will cause it to descend until it reaches Z equals height of a model it touches, instead of Z equals 0. This allows the selected model to stand on another model at its height while in contact at model height level. Finally, when the model stops getting in contact, the selected model starts to get pulled by gravity again until it reaches Z equals 0.

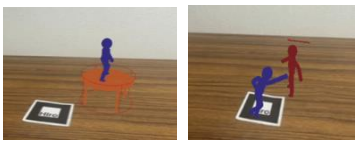
This interaction allows the main model not just to move across the ground level but also to navigate vertically by jumping onto and standing on other models. This means we can stack models on top of each other which is also crucial for 3D interactions, and construction activities.

4.6 Customized interactions

As we can see, there are many possibilities for 3D model interactions. We can create interactions based on our specific needs, for example, if we try to create a fighting game, one common interaction would be “attacking”. We can build a basic attacking mechanism as follows. When the selected model is in contact with another model and the attack key is pressed, the other model will be knocked away at a certain distance and its health will be reduced. If the health of that model falls below zero, that model will be removed from the scene

This is just an example of interaction for specific needs. Therefore, whether for gaming, educational tools, or virtual simulations, 3D models' interactions can be customized and designed to create a dynamic and engaging user experience.

(a) Domain Control (b) Picking and dropping



(c) Stacking on a model (d) Attack a model

Fig 16. Illustrations of the many 3D models interactions

4.7 Model type-based interactions

One of fun way to go with this is to build an interactions base on the type of 3D model. Let’s choose a “door” model as an example. We could do something like, when model contacted a door, we change the environment. Since we will deal with environment changing, let’s switch from detect marker from camera feeds to images, so we can switch images when the selected model contacts model type doors.

We can display 3D models onto images by utilizing a combination of OpenCV, ARToolKit, and OpenGL. Initially, OpenCV processes the image. then, ARToolkit is used to detect markers within the image. Finally, OpenGL is used to draw the 3D models onto the image. By using OpenCV, markers can be detected not only from live camera feeds but also from images.

Now, let’s discuss door type model interaction. When adding a model to the scene, we can specify the model type as for example "isDoor". If the model type is "isDoor", we also need to assign an index related to image switching (for example, index 0 is hallway, and index 1 is bedroom). If the selected model touches another model, we check the type of that model. If it is "isDoor" type, we switch to the image corresponding to the specified index get from the door data.

Fig 17. Door type interaction

4.7.1 AR environments transition through doors

The key focus here is on creating a perception of interconnected environments, where each transition through a door leads to a different yet logically connected space, enhancing the sense of continuity. We can design an image with 3D models to make it look like an organized environment. Once designed, these models can be saved and later reloaded specifically for use with the image they were initially designed for. Fig 18 illustrates the setup of an AR environment aligned with a specific image (in this case, a parking lot). A door in a parking lot image could serve as a gateway to different environments such as inside a building.

Fig 18. Setting up AR environment

Figure 19 shows an AR environment transition flowchart. First, we set up environment data, including images, 3D models' files to load on each image, and door data (door data tells us which environments to switch to). We then initialize the first current environment to start with. The loop starts by detecting a marker from an image of the current environment and loading 3D models file of that current environment. We constantly check models' collision at the run time. If the primary model does not collide with any models, we simply do nothing. But, if the primary model did collide with a model, we check if that model is a type of door. If not, we simply do nothing (or other interactions), but if it is a type of door, which means we need to switch environment. We start by getting index data from that door, then deleting all the models currently loading in the current environment and switching the current environment to new environment corresponding to door data index. After we get a new environment, we simply redo marker detection from the new environment's image, models environment loading, and model's collision checking.

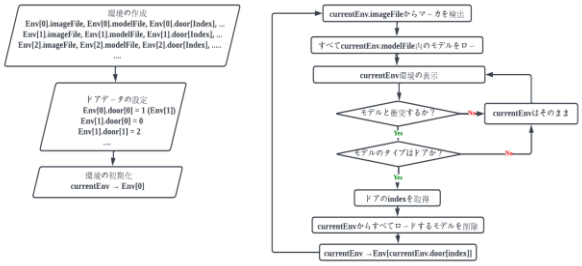


Fig 19. AR environment transition flowchart

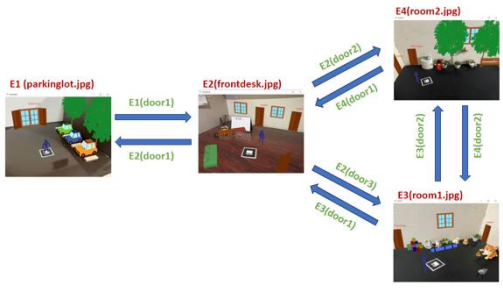


Fig 20. Example of AR environment transition

This approach facilitates seamless transitions between AR settings, enhancing the user experience with diverse environments and paving the way for intricate narratives in interactive storytelling and gaming.

5. Game development

5.1 Game overview

This is a simple game of trying to stop enemies from entering the earth located at the center of an image. The alien's objective is to reach and enter the earth. The player's objective is to stop enemies by eliminating all enemies before they enter the earth. If an enemy entered the earth, the earth's health gradually decreased, the game ends when the earth's health drops below zero. The player wins by killing all the enemies before the earth's health drops below zero.

5.2 3D models' characters design

We can simply design simple characters using primitive shapes like cubes, cylinders, spheres, and cones. Characters in this game include a spaceship, the earth, portals, many types of enemies, and different type of guns ammos.

5.2.1 Spaceship, the earth and portals

Spaceship

The player could manipulate the spaceship by moving it around using keyboard inputs. The spaceship controls which and where projectiles go and shoots with different type of guns to kill enemies.

The earth

The earth has health that dictates the game ending. The earth stays still at one position at the center. Every time enemies reach and enter the earth, the earth's health decreases, and if the earth drops to zero, the player loses.

Portals

Portals are locations where aliens will be generated. We can complicate the game, by putting more and more portals on the scene

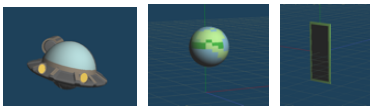


Fig 21. The spaceship, the earth and portals

5.2.2 Enemies

The potential for creating a wide array of enemies in the game is vast, offering numerous opportunities to enhance both challenge and enjoyment. Each enemy can be designed with distinctive attributes, strengths, weaknesses, abilities, and visual designs. For instance, depending on enemy type, they could vary in speed and health levels, adding complexity and depth to the gameplay.

Furthermore, we can program enemies with unique abilities adding an extra layer of complexity and strategy. These abilities can range from simple tactics, such as changing location to evade attacks, to more complicated features like spawning surrounding armies This flexibility in enemy design opens endless possibilities, enabling us to create a rich and diverse gaming experience.

Fig 22. Enemies' characters design

5.2.3 Guns

From the player's side, the design scope for guns in the game is extensive, providing enough scope to enhance the player's experience with a variety of tactical options. Each weapon can be uniquely designed with a set of capabilities and effects. For instance, variations in firing rates and damage impact.

Creative features can be programmed into these weapons to enhance gameplay dynamics. These features could range from simple modifications like projectiles that slow down enemy's speed, to more complex mechanisms such as projectiles that follow targets. The possibilities in gun design allow for the creation of diverse and engaging weapons, each bringing a unique flavor to the player's combat toolkit. This varied weaponry not only adds to the game's excitement but also encourages players to experiment with different combat approaches, making for an enjoyable gaming experience.

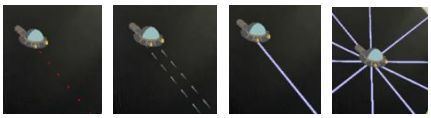


Fig 23. Different type of guns design

5.3 Characters manipulation and interaction

5.3.1 Targeting earth

When enemies' models are generated, we programmed these enemies to move towards the earth model by adjusting their position continuously. When the enemies' models reached and contacted the earth model, the earth health is decreased, and we removed the enemy's model from the scene simulating the enemies enter the earth.

5.3.2 Enemies abilities

In the game's loop, time tracking is crucial for generating enemies at some intervals. As the game progresses, these intervals decrease, making the game more challenging. The generation process involves selecting a random portal already displayed on the scene and generating an enemy at the portal's location. The enemy's movement is directed towards the Earth at the center of the image. Enemy type is also randomized when enemy generation function is called, lending diversity in attributes, strengths, weaknesses, abilities, and visual design. Below are some enemies designs within the game.

Fast and Fragile Enemy: The speed of an enemy in the game is determined by how much its position changes over each frame in the game loop. For this type, the position increment value is set higher compared to other enemies. This is achieved by increasing the value added to its current position towards the target (e.g., earth model) in each iteration of the game loop, making it move faster.

Tank Enemy: In contrast to the fast enemy, this type has a lower position increment value, making its movement slower. Its durability is enhanced by assigning a higher value to its health variable, requiring more hits to be defeated.

Splitting Enemy: When this enemy's health reaches zero, a function is called to generate new enemy models with the same type at its current location. These new models are smaller (scaled down in size) and have reduced health. The function that generates these new models is designed to stop creating more splits once a minimum size is reached.

Teleporting Enemy: This enemy type has an additional variable tracking the damage it receives. Once this damage reaches a specified threshold, the enemy's position is randomly reset to an appropriate new location. After teleporting, the damage variable is reset to zero.

Swarmer Enemy: The primary enemy of this type has a timer or counter that tracks the intervals between spawning new, smaller subordination enemies. Once the counter reaches the set time (e.g., random interval or every 3 seconds), it spawns several smaller enemies' models around it. This creates a dual challenge for the player to target the resilient Queen (primary) while managing the frequent appearance of weaker armies (subordinate).

Stealth Enemy: This enemy is designed with reduced visibility features. By reducing the alpha value of the 3D model, the enemy's transparency is increased, making it more difficult for players to visually detect it against the game background. Its movement towards the target is standard, but its reduced visibility makes it a sneaky threat.

Portal Creator Enemy: This enemy type has a unique behavior pattern. Upon generation, a target location is randomly determined. The enemy moves towards this location, and upon arrival, a function is called to generate a new portal model at that position. This new portal then serves as an additional enemy generation point for other enemies, increasing the game's complexity.

In all cases, the manipulation of the 3D models of enemy's characters is primarily controlled through adjustments to their position vectors within the 3D space of the game, in conjunction with other specific attributes and behaviors defined in the game's code. These manipulations are executed within the game loop, ensuring continuous and dynamic interaction between the player, enemies, and other elements of the game environment.

5.3.3 Shooting mechanisms

In the game, shooting mechanics are implemented to allow the player to defend against incoming enemies. The process begins with the generation of projectiles models, originating from the player character's model. These projectiles models are propelled in the direction the spaceship character is facing, calculated using the spaceship character's rotation angle. Upon activation, each projectile moves consistently forward based on its directional vector, which is determined by the trigonometric functions of the spaceship character's rotation. The game continuously updates the positions of all active projectiles, propelling them at a fixed speed. These projectiles maintain a linear trajectory until they collide with an enemy model or leave the game scene. A collision check is performed between each projectile and enemy model. Upon a successful collision, the enemy's health is reduced by a predefined damage value associated with the projectile type. The game then assesses if the enemy's health has dropped to zero or below, triggering its removal from the scene if it has been effectively 'destroyed'.

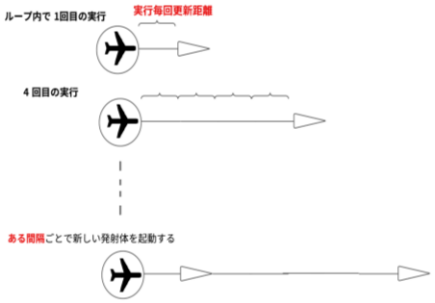


Fig 24. Illustration of shooting mechanism

In addition to the standard shooting mechanics, further enhancements and adjustments have been programmed to introduce a variety of specialized guns, each offering unique strategic advantages.

Fast Guns: These guns accelerate the standard projectile's velocity, achieving rapid target engagement. The code adjustments for Fast Guns involve increasing the update speed and decreasing the interval time at which a next projectile is activated allowing more bullets to travel faster and cover more distance in less time.

Ice Guns: This is realized by programming the projectiles to not only reduce the health of the enemy upon collision but also to temporarily decrease their movement speed.

Follow Guns: The bullets actively adjust their trajectory towards the nearest enemy, calculated using real-time positional data.

Multiple Guns: Expanding the shooting system, Multiple Guns fire multiple projectiles simultaneously from offset positions relative to the player character. This is achieved by creating multiple projectiles, following its own calculated trajectory based on the character's orientation.

Explosive type guns

In the game, we've developed three types of explosive guns, each with unique effects that add depth and variety to combat scenarios.

Explosive Guns: These weapons are designed for area-of-effect damage. Upon hitting a target, they trigger an explosion that affects enemies within a specific radius. This makes them ideal for situations where enemies are clustered together, allowing the player to deal damage to multiple targets with a single shot.

Explosive Freezer Guns: Building on the concept of the Explosive Guns, these add a freezing effect to the explosion. Not only do they damage enemies within the blast radius, but they also stop them from moving. We can simply color the enemies' models that are frozen to be very blue differentiating them from enemies that are not frozen. This freezing effect halts enemy movement, giving the player strategic control over the battlefield and the opportunity to plan their next moves without immediate pressure.

Blackhole Guns: A more advanced and tactical weapon, the Blackhole Gun shoots a temporary black hole. This black hole exerts a pull, drawing nearby enemies towards its center. Despite the type of enemies, if they come into contact with the black hole they are instantly eliminated, making it a powerful tool for controlling large groups of enemies. To maintain game balance, the black hole's duration is time-limited, preventing it from making the game too easy. Its ability to suck in enemies adds a layer of strategy, as players can use it to manipulate enemy positions and clear crowded areas effectively.

Laser gun

The direction of the laser beam is determined by the rotation angle of the player's spaceship character. The game calculates the interaction between the laser beam and enemies. For each enemy, it projects their position onto the direction vector of the laser. This projection helps determine the closest point on the laser line to each enemy. If the distance from the closest point (point D in fig 25) on the laser line to each enemy is within the enemy radius, it means the enemy got hit, therefore enemy's health is reduced by a predefined amount, representing damage inflicted by the laser. Unlike traditional projectiles, the laser's impact is instantaneous along its length, allowing for quick engagement with multiple enemies aligned with the beam. Furthermore, we can optimize this laser gun by adding more laser beams from multiple directions at the same time.

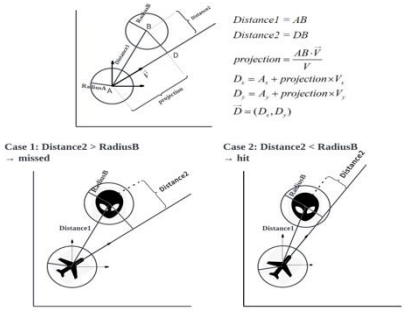


Fig 25. Illustration of directional collision concept

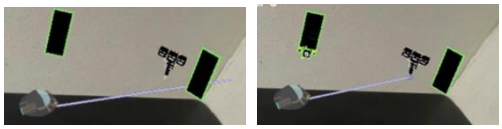


Fig 26. Directional collision

5.3.4 Drop items mechanism

To enhance the interactivity and challenge of the game, a crucial aspect is the implementation of a drop item mechanism. Upon the defeat of enemies, there is a chance it drops various items at their last position. When the player's character model comes into contact with these dropped items, they are collected, adding a layer of engagement to the gameplay. The types of items dropped vary and are chosen based on their usefulness and impact on gameplay. Possible drops include boosts to Earth's health, score increments, and different types of weapons, each adding a tactical advantage or benefit to the player. The likelihood of each item type dropping is carefully decided to balance the game's challenge and reward system.

The drop mechanism encourages players to actively engage with enemies. Additionally, it aids in maintaining the game's difficulty curve, as the acquisition of these items can be crucial for surviving more challenging stages. The integration of this mechanism is designed to ensure a cohesive and dynamic gaming experience, where player actions are consistently rewarded and contribute to the overall progression within the game.

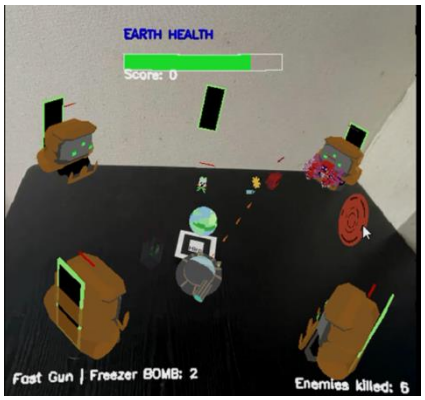


Fig 27. Earth Defender Game

The "Earth Defender" game is an example of 3D model manipulation and interaction within a gaming context. The core of the game' lies in its dynamic use of 3D models – from the diverse array of enemies with unique movements and abilities of weapons, each with distinctive effects and trajectories. The interaction between these models – whether it's a projectile colliding with an enemy, the player navigating a spaceship, or the player collecting drop items– creates a rich and complex interplay of actions and reactions that are both visually engaging and strategically complex.

This game not only showcases the technical skills of 3D modeling but also highlights how these models can be brought to life through intricate programming and creative design. The interactions and behaviors programmed into each model turn them from mere digital objects into key elements of an immersive world, where every movement and collision contribute to the unfolding story of the game. "Earth Defender" is an example of game design, where 3D models are not just visual elements, but active participants in a dynamic, interactive universe.

6. Conclusion

This research journey in the realm of Augmented Reality (AR) has led us to explore the intricate dynamics of 3D model manipulation and interaction within an AR environment. Our exploration, grounded in the use of ARToolKit and complemented by other software like Metasequoia 4, OpenGL, GLMetaseq library, OpenCV has not only highlighted the capabilities of current AR technology but also illuminated the path for future advancements in this field.

We began by understanding the fundamentals of ARToolkit, a powerful tool for developing AR experiences. Our journey then took us through the various stages of 3D model manipulation from manipulation of a single model to manipulation of multiple models. We further delved into the realms of control, where we successfully implemented features such as model selection, deletion, and the concept of manipulating multiple models as a singular entity. We also go through 3D model interactions from collision detection to domain control. Each step revealed more about the potential and challenges of integrating 3D models into AR environments. This model management not only enhanced efficiency but also opened new avenues for creative application within AR spaces.

The pinnacle of our research was the development of the "Earth Defender" AR game, a practical application that showcased our system's capabilities in a real-world scenario. This game, with its array of characters, weapons, and interactive elements, demonstrated the effectiveness of our programming techniques in creating a dynamic and engaging AR experience. The game's design and implementation served as an example of the ability of AR in gaming and its potential for educational and design applications.

In conclusion, I hope this research contributes to the evolving landscape of AR technology. We have demonstrated that with the right tools and creativity, AR can offer much more than just a novel visual experience; it can create interactive, immersive environments with vast potential in various sectors. Our findings pave the way for future research, particularly in enhancing user interaction with AR and overcoming current technological limitations. As AR continues to evolve, it holds the promise of transforming how we interact with our digital and physical worlds, making our interactions more intuitive, engaging, and meaningful.

Reference

[1] Tips for Metasequoia 4
https://www.metaseq.net/en/tips.html

[2] Free 3D Models Resources
https://free3d.com/

[3] Introduction to ARToolKit
http://kougaku-navi.net/ARToolKit/ 2024/01/19

[4] 3D キャラクターが現実世界に誕生! ARToolKit 拡張現実感プログラミング入門, 2008/9/17